Question 5: Version Control Setup, Branching, Merging & GitHub Integration

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: Git Repository Initialization, Staging, Commit History, Branching, Non-Fast-Forward Merge, and Remote GitHub Push

---

1. Overview of Version Control in DevOps

Distributed Version Control Systems (DVCS) like Git provide the single source of truth for all source code, automation scripts, and configuration-as-code manifests. This report demonstrates the complete Git lifecycle executed in Lab 1: repository initialization, file staging, semantic committing, isolated feature branching, non-fast-forward merging, and publishing to a GitHub remote.

---

2. Step-by-Step Version Control Execution

Step 1: Initialize Git Repository

```
# Navigate to workspace and initialize repository with root branch named 'main'
cd "d:\Devops lab 1"
git init -b main
# Output: Initialized empty Git repository in D:/Devops lab 1/.git/
```

Step 2: Author Application Code & Inspect Working Tree

Application assets were created:

- `app.py`: Flask application with `/`, `/health`, and `/api/info` endpoints.
- `test_app.py`: PyTest unit test suite.
- `requirements.txt`: Python package dependencies.
- `Dockerfile`: Multi-stage container packaging definition.
- `deploy.sh`: Automated deployment shell script.
- `Jenkinsfile`: Declarative CI/CD pipeline definition.
- `.gitignore`: Excludes bytecaches, virtual environments, and log files.
- `README.md`: Architecture overview and execution guide.

```
git status
# Shows untracked files ready for staging
```

Step 3: Stage and Commit Initial Codebase

```
git add .
git commit -m "feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline"
# Output:
# [main (root-commit) fb5fb47] feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline
#  8 files changed, 551 insertions(+)
```

```
#  create mode 100644 .gitignore
#  create mode 100644 Dockerfile
#  create mode 100644 Jenkinsfile
#  create mode 100644 README.md
#  create mode 100644 app.py
#  create mode 100644 deploy.sh
#  create mode 100644 requirements.txt
#  create mode 100644 test_app.py
```

Step 4: Branching Strategy & Feature Development

To adhere to DevOps best practices, direct commits to `main` are restricted during active feature development. A feature branch was created:

```
git checkout -b feature/enhance-ui-metrics
# Output: Switched to a new branch 'feature/enhance-ui-metrics'
```

On this branch, a `/metrics` endpoint was added to `app.py`, and corresponding unit tests were added to `test_app.py`.

```
git add app.py test_app.py
git commit -m "feat(metrics): add /metrics monitoring endpoint and test coverage"
# Output:
# [feature/enhance-ui-metrics 1a63744] feat(metrics): add /metrics monitoring endpoint and test coverage
#  2 files changed, 17 insertions(+)
```

Step 5: Merging Feature Branch into Main

To maintain branch provenance and auditability in CI/CD, the merge was executed using `--no-ff` (no fast-forward):

```
git checkout main
git merge --no-ff feature/enhance-ui-metrics -m "merge: Merge branch 'feature/enhance-ui-metrics' into main"
# Output:
# Merge made by the 'ort' strategy.
#  app.py      | 9 +++++++++
#  test_app.py | 8 ++++++++
#  2 files changed, 17 insertions(+)
```

Step 6: GitHub Remote Configuration & Push

```
# Add GitHub remote origin
git remote add origin https://github.com/Akash2347/Devops-Lab-1.git

# Push main branch with upstream tracking
git push -u origin main

# Push feature branch for pull request traceability
git push -u origin feature/enhance-ui-metrics
```

---

3. Git Commit History & Topological Graph

Execution of `git log --graph --oneline --all --decorate`:

```
*   93a862a (HEAD -> main) merge: Merge branch 'feature/enhance-ui-metrics' into main
|\
| * 1a63744 (feature/enhance-ui-metrics) feat(metrics): add /metrics monitoring endpoint and test coverage
```

```
|/
* fb5fb47 feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline
```

---

4. Technical Observations

1. **Branch Protection:** Developing on `feature/*` branches isolates unstable experimental code from breaking the CI/CD pipeline monitoring `main`.
2. **Explicit Merge Commits (`--no-ff`):** Fast-forward merges collapse branch history into a single linear timeline. Using `--no-ff` records the exact merge commit and enables one-command rollbacks (`git revert -m 1 <merge-commit-hash>`).
3. **Semantic Commit Messages:** Following the conventional commit specification (`feat:`, `fix:`, `merge:`) enables automated changelog generation and automated semantic version bumping.